Operating Systems

SRN: **PES1UG25CS836** Date: 16/4/2026

Screenshots: -

1A

```
rohit@ubuntu:~/os-orange-pes-vcs$ ./test_objects
Stored blob with hash: d58213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
Object stored at: .pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
PASS: blob storage
PASS: deduplication
PASS: integrity check

All Phase 1 tests passed.
rohit@ubuntu:~/os-orange-pes-vcs$
```

1B

```
rohit@ubuntu:~/os-orange-pes-vcs$ find .pes/objects -type f
.pes/objects/2a/594d39232787fba8eb7287418aec99c8fc2ecdaf5aaf2e650eda471e566fcf
.pes/objects/d5/8213f5dbe0629b5c2fa28e5c7d4213ea09227ed0221bbe9db5e5c4b9aafc12
.pes/objects/25/ef1fa07ea68a52f800dc80756ee6b7ae34b337afedb9b46a1af8e11ec4f476
rohit@ubuntu:~/os-orange-pes-vcs$
```

2A

```
rohit@ubuntu:~/os-orange-pes-vcs$ make test_tree
gcc -Wall -Wextra -O2 -c tree.c -o tree.o
gcc -o test_tree test_tree.o object.o tree.o -lcrypto
rohit@ubuntu:~/os-orange-pes-vcs$ ./test_tree
Serialized tree: 139 bytes
PASS: tree serialize/parse roundtrip
PASS: tree deterministic serialization

All Phase 2 tests passed.
rohit@ubuntu:~/os-orange-pes-vcs$
```

2B

```
rohit@ubuntu:~/os-orange-pes-vcs$ find .pes/objects -type f
.pes/objects/76/a2601df8dad9dcef1b330b9334993e039d461252b51adbbdd3fa1cc62816c0
.pes/objects/bb/7453659ced8100324e1377f9b4388f5740236bb8be1ffb27d2a74940422963
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
.pes/objects/6b/ea976ffeacb042cfc20bdfd4e9b5d345c2bc83b75e8660df5cef4d52283cda
.pes/objects/5c/4e1e2f5a561dd416ca159203c3259111f64259b7341e2754484f9b8209b0ae
.pes/objects/bc/0c4825558c5d8e032506bfb04b0d29445687a08d947aa84af226abd2d70c0e
.pes/objects/c0/2eea48c148399ddf876b615e71e0bac35523cbf6ecff3766feb0467e284553
.pes/objects/63/09e2c15bfa855a7d7dbfac97f7082b6c4f7a06e0d3b7feccb5e8d59d24c888
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/d8/db362c75ce061b40e5778abaadcbd3b6ad5f6efcbd5bcca809971f45f3a07d
rohit@ubuntu:~/os-orange-pes-vcs$ xxd .pes/objects/76/a2601df8dad9dcef1b330b9334993e039d461252b51adbbdd3fa1cc62816c0
00000000: 7472 6565 2031 3437 0031 3030 3634 3420  tree 147.100644
00000010: 6669 6c65 312e 7478 7400 2cf8 d83d 9ee2  file1.txt.,..=..
00000020: 9543 b34a 8772 7421 fdec b7e3 f3a1 83d3  .C.J.rt!........
00000030: 3763 9025 de57 6db9 ebb4 3130 3036 3434  7c.%.Wm...100644
00000040: 2066 696c 6532 2e74 7874 00bb 7453 659c   file2.txt..tSe.
00000050: ed81 0032 4e13 77f9 b438 8f57 4023 6bb8  ...2N.w..8.W@#k.
00000060: be1f fb27 d2a7 4940 4229 6331 3030 3634  ...'..I@B)c10064
00000070: 3420 6865 6c6c 6f2e 7478 7400 6622 4663  4 hello.txt.f"Fc
00000080: d23e 6f4d 9de9 e2c7 e6d8 7643 05a9 2a38  .>oM......vC..*8
00000090: 30a1 a52d 3d5f 4aa8 007b 5c39            0..-=_J..{\9
```

3A

```
rohit@ubuntu:~/os-orange-pes-vcs$ ./pes init
Initialized empty PES repository in .pes/
rohit@ubuntu:~/os-orange-pes-vcs$ echo "hello" > file1.txt
rohit@ubuntu:~/os-orange-pes-vcs$ echo "hello2" > file2.txt
rohit@ubuntu:~/os-orange-pes-vcs$ ./pes add file1.txt file2.txt
rohit@ubuntu:~/os-orange-pes-vcs$ ./pes status
Staged changes:
  staged:     file1.txt
  staged:     file2.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  untracked:  commit.h
  untracked:  pes.c
  untracked:  tree.c
  untracked:  tree.h
  untracked:  Makefile
  untracked:  index.c
  untracked:  pes.h
  untracked:  test_sequence.sh
  untracked:  commit.c
  untracked:  object.c
  untracked:  test_objects.c
  untracked:  test_tree.c
  untracked:  index.h
  untracked:  .gitignore
```

3B

```
rohit@ubuntu:~/os-orange-pes-vcs$ cat .pes/index
100644 2cf8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4 1776324274 6 file1.txt
100644 bb7453659ced8100324e1377f9b4388f5740236bb8be1ffb27d2a74940422963 1776324278 7 file2.txt
```

4A

```
rohit@ubuntu:~/os-orange-pes-vcs$ ./pes log
commit 6309e2c15bfa855a7d7dbfac97f7082b6c4f7a06e0d3b7feccb5e8d59d24c888
Author: Rohit Pujari PES1UG25CS836
Date:   1776324724

    Add farewell

commit d8db362c75ce061b40e5778abaadcbd3b6ad5f6efcbd5bcca809971f45f3a07d
Author: Rohit Pujari PES1UG25CS836
Date:   1776324684

    Add world

commit bc0c4825558c5d8e032506bfb04b0d29445687a08d947aa84af226abd2d70c0e
Author: Rohit Pujari PES1UG25CS836
Date:   1776324638

    initial commit
```

4B

```
rohit@ubuntu:~/os-orange-pes-vcs$ find .pes -type f | sort
.pes/HEAD
.pes/index
.pes/objects/2c/f8d83d9ee29543b34a87727421fdecb7e3f3a183d337639025de576db9ebb4
.pes/objects/5c/4e1e2f5a561dd416ca159203c3259111f64259b7341e2754484f9b8209b0ae
.pes/objects/63/09e2c15bfa855a7d7dbfac97f7082b6c4f7a06e0d3b7feccb5e8d59d24c888
.pes/objects/66/224663d23e6f4d9de9e2c7e6d8764305a92a3830a1a52d3d5f4aa8007b5c39
.pes/objects/6b/ea976ffeacb042cfc20bdfd4e9b5d345c2bc83b75e8660df5cef4d52283cda
.pes/objects/76/a2601df8dad9dcef1b330b9334993e039d461252b51adbbdd3fa1cc62816c0
.pes/objects/bb/7453659ced8100324e1377f9b4388f5740236bb8be1ffb27d2a74940422963
.pes/objects/bc/0c4825558c5d8e032506bfb04b0d29445687a08d947aa84af226abd2d70c0e
.pes/objects/c0/2eea48c148399ddf876b615e71e0bac35523cbf6ecff3766feb0467e284553
.pes/objects/d8/db362c75ce061b40e5778abaadcbd3b6ad5f6efcbd5bcca809971f45f3a07d
.pes/objects/e0/0c50e16a2df38f8d6bf809e181ad0248da6e6719f35f9f7e65d6f606199f7f
.pes/refs/heads/main
```

4C

```
rohit@ubuntu:~/os-orange-pes-vcs$ cat .pes/refs/heads/main
6309e2c15bfa855a7d7dbfac97f7082b6c4f7a06e0d3b7feccb5e8d59d24c888
rohit@ubuntu:~/os-orange-pes-vcs$ cat .pes/HEAD
ref: refs/heads/main
```

Final

```
rohit@ubuntu:~/os-orange-pes-vcs$ make test-integration
=== Running integration tests ===
bash test_sequence.sh
=== PES-VCS Integration Test ===

--- Repository Initialization ---
Initialized empty PES repository in .pes/
PASS: .pes/objects exists
PASS: .pes/refs/heads exists
PASS: .pes/HEAD exists

--- Staging Files ---
Status after add:
Staged changes:
  staged:     file.txt
  staged:     hello.txt

Unstaged changes:
  (nothing to show)

Untracked files:
  (nothing to show)


--- First Commit ---
Committed: 2645e098b712... Initial commit

Log after first commit:
commit 2645e098b712edbdc71142e3725237167d79c1d44d221cb03a5cedb3859c9bb1
Author: Rohit Pujari PES1UG25CS836
Date:   1776328963

    Initial commit


--- Second Commit ---
Committed: b8b33cf446e1... Update file.txt

--- Third Commit ---
Committed: 33f4d88ed1a2... Add farewell

--- Full History ---
commit 33f4d88ed1a2d8fe716c83c3868e7bff1f47c1bd7d5b0f2fd8c56b385cc62764
Author: Rohit Pujari PES1UG25CS836
Date:   1776328964

    Add farewell

commit b8b33cf446e1593a5046b75f833f4d2f54f41b70be5732fe589ac5250ccf5d58
Author: Rohit Pujari PES1UG25CS836
Date:   1776328963

    Update file.txt

commit 2645e098b712edbdc71142e3725237167d79c1d44d221cb03a5cedb3859c9bb1
Author: Rohit Pujari PES1UG25CS836
Date:   1776328963

    Initial commit


--- Reference Chain ---
HEAD:
ref: refs/heads/main
refs/heads/main:
33f4d88ed1a2d8fe716c83c3868e7bff1f47c1bd7d5b0f2fd8c56b385cc62764

--- Object Store ---
Objects created:
10
.pes/objects/0b/d69098bd9b9cc5934a610ab65da429b525361147faa7b5b922919e9a23143d
.pes/objects/10/1f28a12274233d119fd3c8d7c7216054ddb5605f3bae21c6fb6ee3c4c7cbfa
.pes/objects/26/45e098b712edbdc71142e3725237167d79c1d44d221cb03a5cedb3859c9bb1
.pes/objects/33/f4d88ed1a2d8fe716c83c3868e7bff1f47c1bd7d5b0f2fd8c56b385cc62764
.pes/objects/58/a67ed1c161a4e89a110968310fe31e39920ef68d4c7c7e0d6695797533f50d
.pes/objects/ab/5824a9ec1ef505b5480b3e37cd50d9c80be55012cabe0ca572dbf959788299
.pes/objects/b1/7b838c5951aa88c09635c5895ef7e08f7fa1974d901ce282f30e08de0ccd92
.pes/objects/b8/b33cf446e1593a5046b75f833f4d2f54f41b70be5732fe589ac5250ccf5d58
.pes/objects/d0/7733c25d2d137b7574be8c5542b562bf48bafeaa3829f61f75b8d10d5350f9
.pes/objects/db/07a1451ca9544dbf66d769b505377d765efae7adc6b97b75cc9d2b3b3da6ff

=== All integration tests completed ===
```

Analysis Question Answers: -

Q5.1) A branch in PES-VCS would be stored as a file inside `.pes/refs/heads/` containing the commit hash of the branch tip. To implement `pes checkout <branch>`, the system would first verify that `.pes/refs/heads/ <branch>` exists and read the commit hash from it. Then `.pes/HEAD` must be updated to `ref: refs/heads/<branch>`. After that, the

commit object must be read, its root tree located, and the tree recursively expanded into the working directory so that files and directories match the target branch snapshot.

The complex part is updating the working directory safely. Existing tracked files may need to be overwritten, deleted, or recreated. Directories may need recursive handling. If the user has local modifications, checkout must avoid destroying them. So even though changing the branch pointer is simple, reconciling the working directory with the target tree is the hard part.

Q5.2) To detect a dirty working directory conflict, compare the current working directory against the index first. The index already stores metadata such as path, size, and modification time, and it also stores the blob hash for the staged version. For each tracked file, check whether the current file still matches the index metadata. If metadata differs, the file is potentially modified. For higher confidence, re-read the file, hash it, and compare that hash with the blob hash stored in the index. Then compare the target branch's tree with the current branch's tracked files. If a file differs between branches and the working directory version also differs from the index, checkout must refuse because switching would overwrite uncommitted work. In short, the conflict condition is: the user changed a tracked file locally, and checkout would replace that same path with different content from the target branch.

Q5.3) Detached HEAD means `.pes/HEAD` contains a commit hash directly instead of a branch reference. In that state, new commits are still created normally, but they are not attached to any branch. HEAD moves from commit to commit directly, so those commits can become unreachable once the user checks out a branch again. A user can recover such commits if they still know the commit hash. They could create a new branch reference pointing to that commit by writing the hash into a new file under `.pes/refs/heads/`, then updating HEAD to point to that branch. Recovery works as long as the commit object still exists in the object store and has not been garbage collected.

Q6.1) To delete unreachable objects, start from all live references such as every file in .pes/refs/heads/ and possibly a detached HEAD commit if present. For each referenced commit, recursively mark reachable objects:

- visit the commit object
- mark its tree
- recursively mark all subtrees and blobs referenced by that tree
- if the commit has a parent, visit the parent commit too

A hash set is the right data structure for tracking reachable object IDs efficiently, because membership tests and inserts are close to O(1). After the marking phase, walk through every file in '`.pes/objects/`'. Any object whose hash is not in the reachable set can be deleted.For a repository with 100,000 commits and 50 branches, the number of visited objects depends on sharing, but in the worst case you may traverse nearly all reachable commits, trees, and blobs. That can easily be hundreds of thousands of objects or more. In practice many objects are shared between commits, so the reachable-set check prevents revisiting the same object repeatedly.

Q6.2) Running garbage collection concurrently with commit creation is dangerous because a new commit may reference objects that are not yet protected by a branch reference. For example, a commit operation may first write blob and tree objects, then write the commit object, and only at the end update the branch file. If GC runs in the middle, it may scan references, fail to see the new commit from any branch, and incorrectly conclude that the newly written objects are unreachable. It could then delete objects that the commit operation is about to reference. Git avoids this with careful coordination. Real Git uses techniques such as locks, atomic ref updates, and grace periods via reflogs and pruning rules so that recently created objects are not collected immediately. This prevents GC from racing with ongoing writes and deleting objects that are still in the process of becoming reachable.